



UNIVERSITETI[®]
METROPOLITAN
TIRANA

Course: Object Oriented Programming

Many Objects

Collections, and objects that own other objects

Evis Plaku



One account at a time

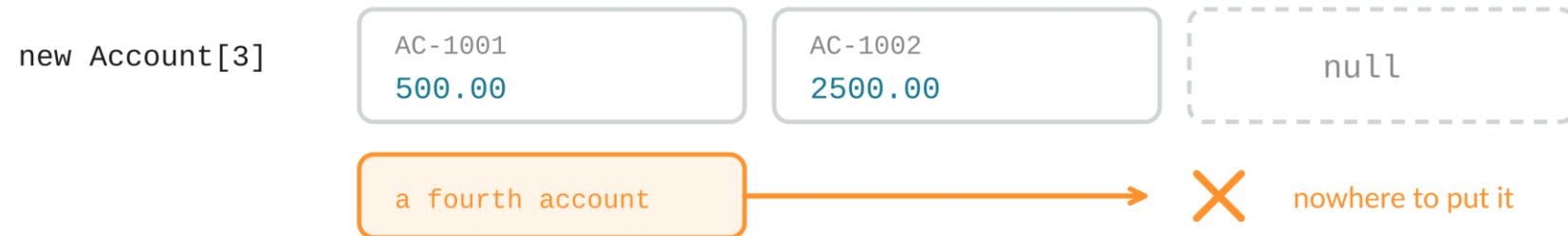
- Three accounts means three variables, named by hand
- A real bank holds a number nobody knew in advance
- You need somewhere to put them that grows

```
Account ada    = new Account("Ada", 500);  
Account alan   = new Account("Alan", 2500);  
Account grace  = new Account("Grace", 1500);
```

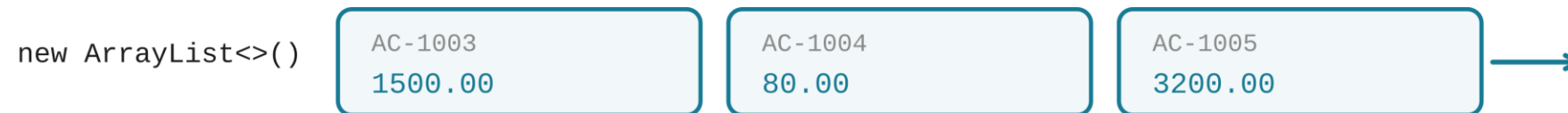
```
// and the fourth? and the fortieth?  
// and the one opened while the  
// program is running?
```

An array decides its size **once, forever**

An array decides its size once, forever



A list grows as you add to it



No size decided up front, no empty slots, and no shifting everything down by hand.



A **List** grows as you add to it

- No size up front, no empty slots, no shifting by hand
- `List<Account>` says this holds accounts and nothing else
- Put a `String` in and the compiler stops you while you type

```
import java.util.ArrayList;
import java.util.List;

List<Account> accounts = new ArrayList<>();

accounts.add(new Account("Ada", 500.00));
accounts.add(new Account("Alan", 2500.00));

accounts.size();           // 2
accounts.get(0);           // Account AC-1001
```



Declare the **promise**, build the **one that keeps it**

- The variable is a List: something you can add to and look through
- The object is an ArrayList: one particular way of keeping that promise
- Swap the implementation later and one line changes, not forty

```
List<Account> accounts
    = new ArrayList<>();

// the promise    the way it is kept

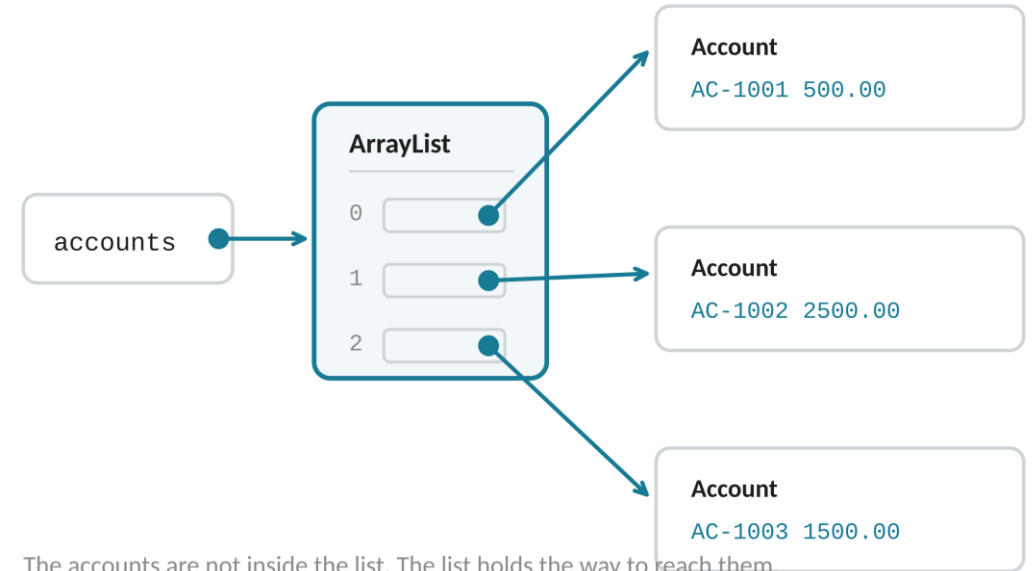
// not this:
// ArrayList<Account> accounts
//     = new ArrayList<>();

// week 8 explains why Java lets you
// write it this way round
```



A list of objects is a list of arrows

- The accounts are not inside the list
- The list holds the way to reach them, nothing more
- This is the week 2 picture with one extra hop





add() copies nothing

- The list and the variable point at the same account
- A deposit through either one is visible through the other
- Convenient now. Less convenient in week 11, writing to a file

```
Account ada = new Account("Ada", 500);  
accounts.add(ada);  
  
ada.deposit(1000.00);  
  
accounts.get(0).getBalance();  
// 1500.0  
  
// predict this before you run it
```



The loop variable is **another arrow**

- So a deposit inside the loop really does change every account
- But pointing it somewhere else changes nothing at all
- Same rule as the method parameter in week 2

```
for (Account account : accounts) {  
    account.deposit(10.00);  
}  
// every account gains 10  
  
for (Account account : accounts) {  
    account = new Account("Nobody", 0);  
}  
// changes nothing
```



Write the loop, and be honest about not finding it

- .equals on the strings, never ==. That was week 2
- null when nothing matched, which puts the work on the caller
- Week 9 offers something better than null

```
static Account findByNumber(  
    List<Account> accounts,  
    String number) {  
  
    for (Account account : accounts) {  
        if (account.getAccountNumber()  
            .equals(number)) {  
            return account;  
        }  
    }  
    return null;  
}
```



contains and remove compare **identity**

They ask each element whether it equals the one you passed

Without instruction, equals still means "is the same object"

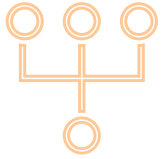
Two accounts with identical data are not the same account

```
accounts.contains(ada);    // true
accounts.contains(twin);  // false

// twin has the same values
// and is a different object

accounts.remove(0);        // by position
accounts.remove(ada);      // by object

// two methods, one name.
// harmless here, a trap with
// List<Integer>
```



Do not rearrange a list while you are walking it

- The loop is reading the list while you change it underneath
- Java notices and throws rather than quietly misbehaving
- Collect first, remove afterwards. Week 12 gives you one line

```
for (Account a : accounts) {  
    if (a.getBalance() == 0) {  
        accounts.remove(a);  
    }  
}  
// ConcurrentModificationException  
  
List<Account> empty = new ArrayList<>();  
for (Account a : accounts) {  
    if (a.getBalance() == 0) empty.add(a);  
}  
accounts.removeAll(empty);
```



List.of gives you a list nobody can extend

Last week's immutability idea, applied to a collection

Use it for fixed sets of values. new ArrayList for anything that grows

Call add on it and it throws, which is the point

```
List<String> accountTypes = List.of(
    "CHECKING", "SAVINGS", "DEPOSIT");

accountTypes.add("CRYPTO");
// UnsupportedOperationException

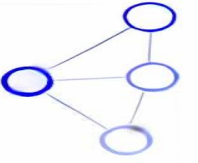
// a list of things that were decided
// once should not be a list anyone
// can extend
```



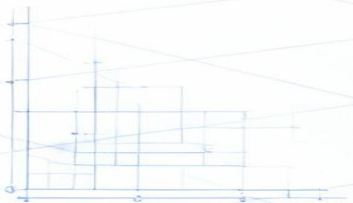
A list of objects is a list of arrows

- The objects live outside it, exactly where new created them
- Adding one to a list does not copy it
- Next: giving that list an owner who keeps it to itself

```
accounts.add(ada);  
// one account, two ways to reach it  
  
for (Account a : accounts) { ... }  
// the loop variable is a third  
  
// every arrow leads to the same  
// object, and changing it through  
// any of them changes it for all
```



Objects that own other objects





A correction first: one fact, one place

- Account stores ownerName. Customer is about to store it too
- Two places holding one fact will eventually disagree
- Change the name on the customer and the accounts show the old one

```
public class Account {  
  
    private final String accountNumber;  
    private double balance;  
  
    // ownerName has left this class  
  
    public Account(double openingBalance) {  
        ...  
    }  
}
```



private, **final**, and created **right there**

- private, so nothing outside can touch it
- final, so the customer always has exactly this list
- Created where it is declared, so no customer exists without one

```
public class Customer {  
  
    private static int lastIdIssued = 1000;  
  
    private final String customerId;  
    private String name;  
  
    private final List<Account> accounts  
        = new ArrayList<>();  
  
}  
  
// what is IN the list changes.  
// WHICH list it is does not.
```

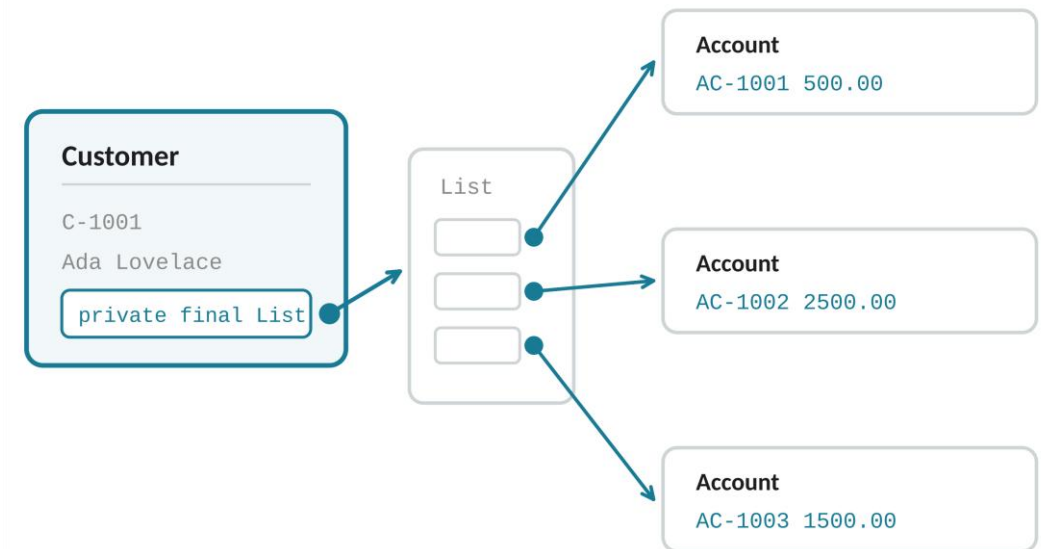


The customer owns the list. Nothing else can reach it

- openAccount is the only door in
- The accounts sit outside the customer, as they always did
- This is the full canonical picture, all arrows visible

openAccount is the only door in

Nothing outside can add to that list, or take from it.



The customer owns the list. The list holds arrows. Neither is ever handed out.



The customer **makes** the account rather than accepting one

- So it can never hold an account belonging to somebody else
- Returning it is fine. The account is not secret. The list is
- closeAccount refuses while there is still money in it

```
public Account openAccount(double opening) {  
    Account account = new Account(opening);  
    accounts.add(account);  
    return account;  
}  
  
public boolean closeAccount(String number) {  
    Account account = findAccount(number);  
    if (account == null) return false;  
    if (account.getBalance() > 0) return false;  
    accounts.remove(account);  
    return true;  
}
```



Never hand out **your own list**

- return accounts gives the caller your actual list
- They can clear it, or add an account belonging to somebody else
- Every guarantee the class made is gone, in one line

```
// do not write this
public List<Account> getAccounts() {
    return accounts;
}

// write this
public List<Account> getAccounts() {
    return List.copyOf(accounts);
}
```



The **list** is new. The **accounts** are not

- Nobody can change which accounts this customer has
- Somebody with the copy can still deposit into one of them
- That is correct. You protected the part that was yours

```
return accounts;
```

the caller gets your actual list
`accounts.clear();`
it works
the customer now has none

```
return List.copyOf(accounts);
```

the caller gets a separate list
`accounts.clear();`
throws. yours is untouched
the customer still has three

Be precise about what got copied

the copy
a new list



the same accounts
`deposit()` still lands on them

You protected which accounts the customer has. That was the part that was yours to protect.



Coordinate. Do not reach in

- The wrong version has Customer checking the balance itself, and it needs a `setBalance` that week 3 refused to build

Reaching in



Asking



The customer does no arithmetic and knows nothing about balance rules.



Find the right account, and ask it

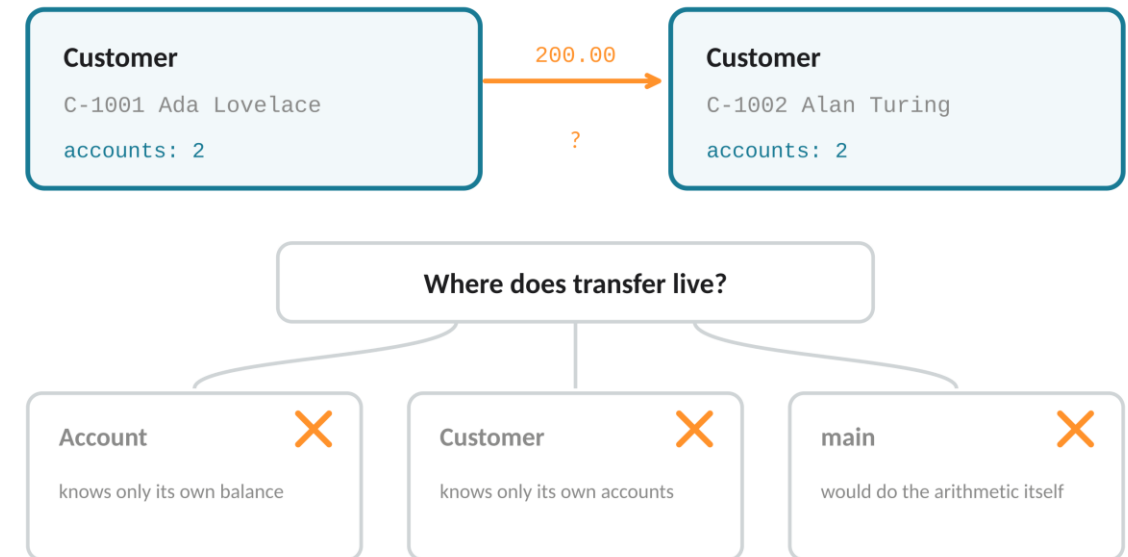
- No arithmetic here, and no balance rules here
- If the overdraft policy changes next year, this does not change
- getTotalBalance is calculated, never stored. Two facts would disagree

```
public boolean withdrawFrom(  
    String number, double amount) {  
  
    Account account = findAccount(number);  
    if (account == null) {  
        return false;  
    }  
    return account.withdraw(amount);  
}
```



Ada wants to send 200 to Alan

- Account knows only its own balance
- Customer knows only its own accounts
- Nobody owns both, and that is where Module 2 starts



Nobody owns both accounts. That is the problem Module 2 opens with.



Questions with no answer to look up

- `getAccounts` returns a copy, yet balances can still change. Hole or correct?
- `closeAccount` refuses when funded. Where else could that rule have lived?
- `withdrawFrom` returns false for two very different reasons. Does it matter?

Don't lend what you can't lose.

That's friendship in life, and your list in OOP

